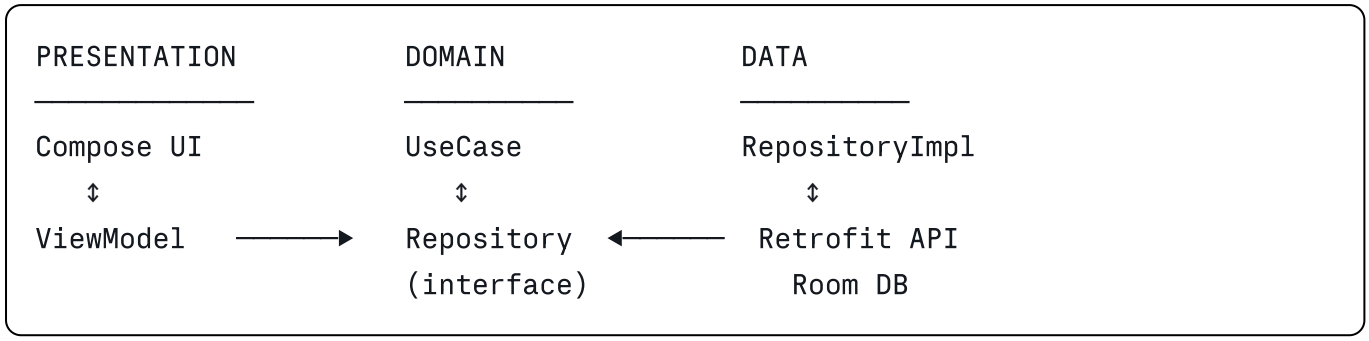
MVVM + Clean Architecture — Minimal Version

The Idea in One Picture



Arrows point **into** Domain. Domain depends on nothing.

The Three Layers

Layer	Contains	Knows about Android?
Presentation	Compose screens, ViewModel, UI state	Yes
Domain	UseCase, Repository *interface*, models	**No — pure Kotlin**
Data	Repository *implementation*, Retrofit, Room, DTOs	Yes

The one rule: Domain has zero imports from Presentation or Data. Everything else follows from that.

Example: Login Screen

DOMAIN

Model

```kotlin
data class User(
    val id: String,
    val email: String,
    val name: String
)
```

Repository interface — the contract lives here, not in Data.

```kotlin
interface AuthRepository {
    suspend fun login(email: String, password: String): User
}
```

UseCase — one job, business rules only.

```kotlin
class LoginUseCase(
    private val repository: AuthRepository
) {
    suspend operator fun invoke(email: String, password: String): User {
        return repository.login(email.trim().lowercase(), password)
    }
}
```

DATA

API

```kotlin
interface AuthApi {
    @POST("auth/login")
    suspend fun login(@Body request: LoginRequest): LoginResponse
}

data class LoginRequest(val email: String, val password: String)

data class LoginResponse(val userId: String, val email: String, val name: Strin
```

Mapper — converts the network shape into the domain model.

```kotlin
fun LoginResponse.toDomain() = User(
    id = userId,
    email = email,
    name = name
)
```

Repository implementation — implements the Domain interface.

```kotlin
```

```kotlin
class AuthRepositoryImpl(
    private val api: AuthApi
) : AuthRepository {

    override suspend fun login(email: String, password: String): User {
        val response = api.login(LoginRequest(email, password))
        return response.toDomain()
    }
}
```

PRESENTATION

UI state — one immutable data class holding everything the screen needs.

kotlin

```kotlin
data class LoginUiState(
    val email: String = "",
    val password: String = "",
    val isLoading: Boolean = false,
    val error: String? = null,
    val isSuccess: Boolean = false
)
```

ViewModel

kotlin

```kotlin
@HiltViewModel
class LoginViewModel @Inject constructor(
    private val loginUseCase: LoginUseCase
) : ViewModel() {

    private val _uiState = MutableStateFlow(LoginUiState())
    val uiState: StateFlow<LoginUiState> = _uiState.asStateFlow()

    fun onEmailChange(value: String) {
        _uiState.update { it.copy(email = value, error = null) }
    }

    fun onPasswordChange(value: String) {
        _uiState.update { it.copy(password = value, error = null) }
    }

    fun login() {
        if (_uiState.value.isLoading) return

        viewModelScope.launch {
            _uiState.update { it.copy(isLoading = true, error = null) }

            try {
                loginUseCase(_uiState.value.email, _uiState.value.password)
                _uiState.update { it.copy(isLoading = false, isSuccess = true)
            } catch (e: Exception) {
                _uiState.update {
                    it.copy(isLoading = false, error = "Login failed. Please tr
                }
            }
        }
    }
}
```

Compose screen

kotlin

```
@Composable
fun LoginScreen(
    viewModel: LoginViewModel = hiltViewModel(),
    onLoginSuccess: () -> Unit
) {
    val state by viewModel.uiState.collectAsStateWithLifecycle()

    LaunchedEffect(state.isSuccess) {
        if (state.isSuccess) onLoginSuccess()
    }

    Column(
        modifier = Modifier.fillMaxSize().padding(24.dp),
        verticalArrangement = Arrangement.Center
    ) {
        OutlinedTextField(
            value = state.email,
            onValueChange = viewModel::onEmailChange,
            label = { Text("Email") },
            enabled = !state.isLoading,
            modifier = Modifier.fillMaxWidth()
        )

        Spacer(Modifier.height(12.dp))

        OutlinedTextField(
            value = state.password,
            onValueChange = viewModel::onPasswordChange,
            label = { Text("Password") },
            visualTransformation = PasswordVisualTransformation(),
            enabled = !state.isLoading,
            modifier = Modifier.fillMaxWidth()
        )

        Spacer(Modifier.height(20.dp))

        Button(
            onClick = viewModel::login,
            enabled = !state.isLoading,
            modifier = Modifier.fillMaxWidth()
        ) {
            Text(if (state.isLoading) "Please wait..." else "Login")
        }

        state.error?.let {
            Spacer(Modifier.height(12.dp))
```

```
            Text(it, color = MaterialTheme.colorScheme.error)
        }
    }
}
```

Hilt Wiring

kotlin

```kotlin
@Module
@InstallIn(SingletonComponent::class)
abstract class AppModule {

    @Binds
    abstract fun bindAuthRepository(impl: AuthRepositoryImpl): AuthRepository
}
```

`@Binds` for interface → implementation. `@Provides` when you have to construct the object yourself (Retrofit, Room).

The Data Flow

```
User taps Login
   ↓
Compose calls viewModel.login()
   ↓
ViewModel sets isLoading = true
   ↓
ViewModel calls loginUseCase(email, password)
   ↓
UseCase calls repository.login(...)          ← Domain interface
   ↓
AuthRepositoryImpl calls api.login(...)      ← Data implementation
   ↓
Response DTO → toDomain() → User
   ↓
Back up through UseCase → ViewModel
   ↓
ViewModel sets isSuccess = true
   ↓
Compose observes state, navigates away
```

Package Structure

```
com.example.app
├── domain
│   ├── model/User.kt
│   ├── repository/AuthRepository.kt
│   └── usecase/LoginUseCase.kt
├── data
│   ├── remote/AuthApi.kt
│   ├── remote/dto/LoginResponse.kt
│   ├── mapper/UserMapper.kt
│   └── repository/AuthRepositoryImpl.kt
└── presentation
    └── login/
        ├── LoginScreen.kt
        ├── LoginViewModel.kt
        └── LoginUiState.kt
```

Why Bother — The Payoff

Testing. `LoginUseCase` is pure Kotlin, so a test needs no Android runtime:

kotlin

```kotlin
@Test
fun `login returns user`() = runTest {
    val repository = mockk<AuthRepository>()
    coEvery { repository.login(any(), any()) } returns User("1", "a@b.com", "Te

    val user = LoginUseCase(repository)("A@B.com", "password")

    assertEquals("a@b.com", user.email)
}
```

Swapping implementations. Move from REST to GraphQL: write a new `AuthRepositoryImpl`, change one line in the Hilt module. Domain and Presentation are untouched.

Thin ViewModel. Business rules sit in UseCases, so the ViewModel only manages UI state.

Interview Answers

"Explain MVVM with Clean Architecture."

> Three layers. Presentation is Compose plus a ViewModel exposing UI state as a StateFlow. Domain holds UseCases and the Repository interface, with no Android

> dependencies. Data implements that interface using Retrofit and Room. Dependencies point inward toward Domain, so Domain stays testable with plain JUnit.

"Why does the Repository interface live in Domain?"

> Dependency inversion. Domain defines what it needs; Data supplies it. That way Domain never depends on Retrofit or Room.

"Isn't a UseCase overkill for a one-line call?"

> Sometimes, but it's the place validation, caching policy, or combining multiple repositories lands later. Keeping it consistent beats saving one file.

"Why StateFlow over LiveData?"

> StateFlow is pure Kotlin so it works in any layer, and it has richer operators. Use `collectAsStateWithLifecycle()` in Compose to get the lifecycle safety LiveData gave you for free.

"What survives a rotation?"

> The ViewModel and everything in its state. Compose `remember` does not — that's why the email and password live in `LoginUiState`, not in the composable.